# Plan de travail en parallèle

Assistant documentaire arabe – français – anglais (RAG, fine-tuning, MCP, évaluation)

Principe : chaque personne construit un mini-système complet de bout en bout. Personne n'attend le travail d'une autre.

## 1. Idée de la répartition horizontale

Au lieu de répartir le projet par composant (une personne fait le RAG, une autre le MCP…), on le répartit par **tranche de langue**. Chaque personne réalise **toutes les étapes** (ingestion, RAG, évaluation, fine-tuning, serveur MCP) sur sa propre version du système, avec ses propres questions. Ainsi :

- chacun **apprend tout** : RAG, fine-tuning, MCP et évaluation ;
- chacun **travaille seul** : son code, ses données dérivées, sa collection vectorielle et ses résultats lui appartiennent ;
- les trois versions sont comparables, car elles suivent le même contrat (section 2) et partent des **mêmes documents**, ce qui facilite l'intégration finale.

Ce plan est écrit pour **3 personnes**. Pour 2 personnes, voir la section 9.

## 2. Contrat commun (jour 1, environ 2 heures, tous ensemble)

C'est la seule étape faite ensemble. Une fois écrit, il est **gelé** : personne ne le modifie sans l'accord des deux autres. Il sert à ce que chaque travail reste compatible sans dépendre des autres.

| Élément | Ce qui est décidé |
|---|---|
| Corpus | Corpus mixte de 10 à 12 documents publics, avec leurs liens. **3 à 4 documents parallèles** (la même publication en arabe, français et anglais, par exemple de l'ONU ou de l'OMS) et **6 à 8 documents dans une seule langue**, sur des sujets proches, qui servent de documents distracteurs. Il n'est pas nécessaire que chaque document existe en trois langues. **Les trois personnes utilisent exactement les mêmes documents** : liste figée dans `contract/corpus.csv` (titre, langue, lien, date de téléchargement, empreinte SHA-256 du fichier). **Une seule personne** exécute le script de téléchargement et partage ensuite le dossier `data/` avec les deux autres, en privé : les pages de l'OMS changent quand une fiche est mise à jour, donc des téléchargements séparés ne donnent pas toujours le même fichier. Le dépôt public ne contient que la liste et le script, pas les documents. |
| Domaine du corpus | **Un seul domaine pour toute l'équipe : santé publique** (publications de l'OMS et de l'ONU). Critères de choix des documents : publications officielles dans les langues concernées, documents web ou PDF avec texte sélectionnable (pas de scan), licence de réutilisation vérifiée pour chaque document, documents courts et stables. Le README précise que l'assistant répond à partir des documents et ne donne pas de conseil médical. |
| Schéma d'un passage (chunk) | Champs obligatoires : `doc_id`, `langue`, `page_ou_section`, `version`, `texte`. Le schéma est écrit dans `contract/chunk_schema.json`. |
| Format des questions | Fichier JSONL : `question`, `langue_question`, `doc_id_attendu`, `page_ou_section_attendue`, `type` (même langue ou translingue). Une question **translingue** doit avoir sa réponse dans un document d'une **autre langue** que la question, sans version parallèle dans la langue de la question ; sinon elle ne teste pas la recherche entre langues. |
| Outils MCP | Mêmes signatures pour tous : `ingest_document`, `search_by_language`, `answer_with_citations` (décrites dans `contract/mcp_tools.json`). |
| Métriques | Recall@5, Recall@10 et MRR, calculées de la même façon. |
| Modèle de base | Un même modèle d'embeddings multilingue comme point de comparaison commun, fixé dans `contract/params.yaml`. |
| Règles du dépôt | Un dossier par personne (`ar/`, `fr/`, `en/`) et un dossier `contract/` en lecture seule. Aucune importation de code entre ces dossiers (seul `integration/` le fait, le dernier jour). Pas de clé dans Git : un fichier `.env.example` sans secret, et `.env` dans `.gitignore` dès le premier commit. Structure, commandes et workflow Git : section 3. |

## 3. Architecture commune et processus de travail

Objectif : les trois dossiers ont **exactement la même structure, les mêmes noms de fichiers, les mêmes fonctions et les mêmes commandes**. Seul le contenu change (langue, variante de recherche, expérience de fine-tuning). Le merge final se réduit alors à ajouter un dossier `integration/`.

### 3.1 Arborescence du dépôt (créée au jour 1 par une seule personne, puis gelée)

```
projet-rag-multilingue/
|-- contract/                  lecture seule, gele au jour 1
|   |-- corpus.csv             liste figee des documents
|   |-- chunk_schema.json      schema d'un passage
|   |-- question_schema.json   schema d'une question d'evaluation
|   |-- results_schema.json    schema de results.json
```

```
|   |-- mcp_tools.json          signatures des 3 outils MCP
|   |-- params.yaml             modele de base, k, graine, decoupage de base
|   `-- validate.py             verifie qu'un dossier respecte le contrat
|-- data/                       documents sources (hors Git, partages en prive)
|-- ar/   fr/   en/             un dossier par personne, structure identique :
|   |-- src/
|   |   |-- ingest.py           lecture, langue, normalisation, decoupage
|   |   |-- index.py            embeddings et collection vectorielle
|   |   |-- retrieve.py         recherche (variante de la personne)
|   |   |-- generate.py         reponse avec citations
|   |   |-- evaluate.py         Recall@5, Recall@10, MRR
|   |   |-- finetune.py         experience de fine-tuning
|   |   `-- mcp_server.py       les 3 outils MCP
|   |-- configs/config.yaml     seul fichier ou la personne regle ses choix
|   |-- eval/questions.jsonl    ses 20 questions
|   |-- results/results.json    ecrit par evaluate.py, jamais a la main
|   |-- notebooks/              exploration, sans sorties enregistrees
|   |-- tests/                  tests minimaux
|   |-- requirements-extra.txt  dependances propres a la personne
|   |-- .env.example
|   `-- README.md
|-- integration/                cree le dernier jour, ensemble
|   |-- compare.py              tableau comparatif des trois results.json
|   |-- router.py               choix de la collection selon la langue
|   `-- mcp_server.py           serveur MCP final
|-- .github/workflows/ci.yml    controles automatiques sur chaque PR
|-- .github/CODEOWNERS          proprietaire de chaque dossier
|-- .gitignore  .gitattributes  .pre-commit-config.yaml
|-- Makefile  requirements.txt  README.md
```

### 3.2 Interfaces identiques (mêmes noms, mêmes signatures dans les trois dossiers)

| Module | Fonction | Rôle |
|---|---|---|
| `ingest.py` | `ingest_document(path, doc_id)` → liste de chunks | Retourne des passages conformes à `chunk_schema.json`. Ajouter un document ne recalcule pas les autres. |
| `index.py` | `build_index(chunks, config)` | Crée ou met à jour la collection `docs_ar`, `docs_fr` ou `docs_en`. |
| `retrieve.py` | `search(query, k=10, langue=None)` | Liste de passages avec `doc_id`, `page_ou_section`, `score`, `texte`. |
| `generate.py` | `answer(query, k=5)` | Dictionnaire `reponse` + `citations` (document et page). |
| `evaluate.py` | `run_eval(questions_path, config)` | Calcule les métriques et écrit `results/results.json`. |
| `finetune.py` | `run_finetune(config)` | Entraîne, évalue avant/après, enregistre dans `results.json`. |
| `mcp_server.py` | `ingest_document`, `search_by_language`, `answer_with_citations` | Simples enveloppes autour des fonctions ci-dessus. |

### 3.3 Mêmes commandes pour tous (Makefile racine)

La cible s'applique à un dossier avec `P=ar`, `P=fr` ou `P=en`.

| | |
|---|---|
| `make setup P=ar` | Crée l'environnement et installe `requirements.txt` puis `requirements-extra.txt`. |
| `make ingest P=ar` | Lit `data/`, produit les passages dans `ar/data_derived/`. |
| `make index P=ar` | Construit la collection vectorielle de la personne. |
| `make eval P=ar` | Évalue la base et écrit `ar/results/results.json`. |
| `make finetune P=ar` | Lance l'expérience de fine-tuning et complète `results.json`. |
| `make serve P=ar` | Démarre le serveur MCP du dossier. |
| `make check P=ar` | Lance `contract/validate.py`, les tests et le formatage. À passer avant chaque PR. |

### 3.4 Reproductibilité : mêmes règles pour tous

— **Une seule version de Python** et un seul `requirements.txt` aux versions épinglées ; les besoins propres à une personne vont dans son `requirements-extra.txt`.

— **Graine fixe** lue dans `contract/params.yaml`, jamais écrite en dur dans le code.

— Tous les choix de la personne (variante, taille des passages, hyperparamètres) sont dans `configs/config.yaml`, pas dans le code.

— Chemins relatifs uniquement ; les modèles fine-tunés et la base vectorielle restent **hors Git** (`.gitignore`), mais le nom du modèle et son empreinte sont notés dans `results.json`.

— Notebooks versionnés **sans sorties** (`nbstripout` dans `pre-commit`), pour éviter des conflits énormes.

### 3.5 Contenu de `results.json` (même schéma pour les trois)

Champs obligatoires : `personne`, `langue_questions`, `modele_base`, `variante`, `commit`, `date`, `graine`, `n_questions`, un bloc `base` et un bloc `finetune` contenant chacun `recall@5`, `recall@10`, `mrr` (au total, et séparés pour les questions de même langue et translingues), et la liste `echecs` (identifiants des questions mal traitées). Cela permet à `integration/compare.py` de produire le tableau comparatif sans adaptation.

### 3.6 Workflow Git

— `main` **est protégée** : aucune modification directe, tout passe par une pull request relue par **au moins une autre personne**.

— **Branches** : `feat/ar-...`, `feat/fr-...`, `feat/en-...` ; `contract/...` pour tout changement du contrat (accord des trois exigé) ; `integration/...` le dernier jour.

— **Commits** préfixés : `[ar]`, `[fr]`, `[en]`, `[contract]`, `[integration]`. Une PR ne touche que le dossier de son auteur.

— **CODEOWNERS** : `ar/` appartient à A, `fr/` à B, `en/` à C, `contract/` et `integration/` aux trois.

— **Fichiers communs** (`Makefile`, `requirements.txt`, `.gitignore`, `README.md` racine) : modifiés seulement par une PR courte, séparée, annoncée aux autres. Un besoin personnel va dans le dossier de la personne.

— Avant chaque PR : `git pull -rebase origin main`, puis `make check P=xx`.

— **CI** (`ci.yml`) : exécute `validate.py`, les tests, le formatage, et refuse tout fichier `.env`, toute clé, tout notebook avec sorties.

### 3.7 Jalons de fusion (le même rythme pour tous)

| Jalon | Ce qui doit être fusionné dans `main` | Condition |
|---|---|---|
| Fin jour 1 | `contract/`, squelette des trois dossiers, Makefile, CI | Les trois relisent et valident. |
| Fin semaine 1 | Ingestion, RAG de base, 20 questions | `make check` vert pour chacun. |
| Fin semaine 2 | Évaluation, fine-tuning, `results.json` complet | `results.json` conforme au schéma. |
| Fin semaine 3 | Serveur MCP, démonstration, README du dossier | Trois outils fonctionnels. |
| Dernier jour | `integration/` : comparaison, routeur, serveur MCP final | Revue commune. |

### 3.8 Ce que vérifie `validate.py` (« terminé » veut dire que ces contrôles passent)

- ☐ La structure du dossier est exactement celle de la section 3.1.
- ☐ Les passages produits respectent `chunk_schema.json`.
- ☐ `questions.jsonl` contient au moins 20 questions, dont au moins 8 translingues, conformes à `question_schema.json`.
- ☐ `results.json` respecte `results_schema.json`.
- ☐ Le serveur MCP expose les 3 outils avec les signatures de `mcp_tools.json`.
- ☐ Aucun import vers un autre dossier de personne.

## 4. Qui fait quoi : trois tranches indépendantes

Chaque personne ingère **tout le corpus commun** (les mêmes documents, dans les trois langues) avec son propre code. La différence : la langue de ses questions et l'expérience qu'elle explore.

| | Personne A | Personne B | Personne C |
|---|---|---|---|
| **Langue des questions** | Arabe | Français | Anglais |
| **Dossier** | `ar/` | `fr/` | `en/` |
| **Collection vectorielle** | `docs_ar` | `docs_fr` | `docs_en` |
| **Variante de recherche** | Dense multilingue | Hybride (mots-clés + dense) | Dense avec deux découpages comparés (par page, par section) |
| **Expérience de fine-tuning** | Embeddings sur des paires question–passage en arabe | Reranker (cross-encoder) sur passages pertinents et non pertinents | Embeddings avec négatifs difficiles, comparés à la base |
| **Questions d'évaluation** | 20 en arabe | 20 en français | 20 en anglais |
| **Serveur MCP** | 3 outils sur `docs_ar` | 3 outils sur `docs_fr` | 3 outils sur `docs_en` |

Les variantes sont des suggestions : l'important est que chacun compare son expérience à la **base commune**.

## 5. Planning identique pour tous (3 semaines)

| Période | Travail de chacun, en parallèle |
|---|---|
| Jour 1 | Contrat commun et squelette du dépôt (section 3), ensemble, puis gel. |
| Semaine 1 | **Ingestion et RAG de base :** lecture des PDF, détection de la langue, normalisation (en arabe : diacritiques et variantes de lettres), découpage avec métadonnées, indexation, recherche et réponse avec citations de page. Rédaction des 20 questions d'évaluation, dont au moins 8 **translingues** (question dans sa langue, réponse trouvée uniquement dans un document d'une autre langue, voir le contrat). |
| Semaine 2 | **Évaluation puis fine-tuning :** Recall@k et MRR de la base, une expérience de fine-tuning, comparaison avant/après, avec les cas d'échec. |
| Semaine 3 | **Serveur MCP, démonstration, README :** les 3 outils, un client de démonstration, et un tableau de résultats avec les limites. |
| Dernier jour | **Intégration (ensemble) :** voir section 7. |

## 6. Livrables de chaque personne (liste de contrôle)

- ☐ Pipeline d'ingestion avec métadonnées, qui gère l'ajout d'un nouveau document sans tout recalculer.
- ☐ RAG avec réponses citant le document et la page.
- ☐ 20 questions d'évaluation (dont 8 translingues) et leurs résultats : Recall@5, Recall@10, MRR.
- ☐ Une expérience de fine-tuning comparée à la base, avec le tableau avant/après.
- ☐ Un serveur MCP avec 3 outils fonctionnels et une démonstration avec un client.
- ☐ Un README : installation, résultats, limites. Aucune clé ni fichier `.env` dans le dépôt.
- ☐ `make check` sans erreur et PR fusionnée à chaque jalon (section 3.7).

Au total, les trois personnes produisent 60 questions d'évaluation, au-delà du minimum de 50.

## 7. Intégration (dernier jour)

Chaque dossier fonctionne seul et suit la même structure, donc l'intégration est légère :

1. **Tableau comparatif** des trois tranches avec `integration/compare.py` : il lit les trois `results.json` (même schéma), mêmes métriques, mêmes modèles de base, résultats par langue de question.
2. **Serveur MCP final** (`integration/mcp_server.py`) : un serveur qui choisit la collection selon la langue (`router.py` et `search_by_language`), en appelant les mêmes fonctions `search` et `answer` de chaque dossier, avec la meilleure variante de recherche retenue d'après les résultats.
3. **Rapport commun** : résultats, ce qui a marché, ce qui n'a pas marché, limites. Les résultats sont publiés tels quels, même si un fine-tuning n'améliore pas la base.

## 8. Ce que chacun apprend

| | A | B | C |
|---|---|---|---|
| Ingestion et métadonnées | ✓ | ✓ | ✓ |
| RAG et citations | ✓ | ✓ | ✓ |
| Évaluation (Recall@k, MRR) | ✓ | ✓ | ✓ |
| Fine-tuning | ✓ | ✓ | ✓ |
| Serveur MCP | ✓ | ✓ | ✓ |
| Git, CI et revue de code | ✓ | ✓ | ✓ |

## 9. Si vous êtes seulement 2

Gardez la même méthode avec deux langues de questions (par exemple arabe et français), un modèle de base commun, et un troisième jeu de questions en anglais écrit à tour de rôle. Chacun garde ses deux expériences de fine-tuning à comparer. Gardez la même architecture : deux dossiers au lieu de trois, le troisième jeu de questions étant ajouté dans `contract/`. Pour une personne seule, réalisez les trois tranches l'une après l'autre en gardant le même contrat.

## 10. Règles pour rester indépendants

— Si un besoin touche le contrat commun, on le signale aux autres et on le modifie ensemble. Sinon, chacun avance seul.

— Chaque personne travaille dans sa branche Git et dans son dossier. Les fusions ne portent que sur les dossiers de leur auteur (section 3).

— Aucune importation entre `ar/`, `fr/` et `en/` ; seul `integration/` importe leurs fonctions, le dernier jour.

— Chacun utilise ses propres clés d'API, stockées dans son `.env` local.

— Les résultats sont enregistrés dans un fichier versionné par personne (`results.json`), jamais modifié par les autres.